

APUNTES TEORIA

Sucesiones · Series · Induccion Matematica

Algebra I · APU · Facultad de Ingenieria · UNJu · 2026

Basado en: Rosen, K. (2004). Matematica Discreta y sus Aplicaciones.
Capitulos 2.4 (Sucesiones y Sumatorias) y 5.1-5.2 (Induccion Matematica)

- | | |
|---|--|
| 1 | SUCESIONES
Definicion · Vocabulario · Tipos de termino general |
| 2 | TIPOS DE SUCESION
Aritmetica · Geometrica · Cuadratica · Cubica · Recursiva |
| 3 | SERIES / SUMATORIAS
Notacion Sigma · Propiedades · Doble sumatoria · Productoria |
| 4 | INDUCCION
Principio PMI · Los 5 pasos · Vocabulario · Patrones |
| 5 | VOCABULARIO CLAVE
Glosario bilingue con analogias de programacion |

PARTE 1 — SUCESIONES

1.1 Que es una Sucesion

■ Rosen — Cap. 2.4, Definicion 1 — pagina 161

DEFINICION — Sucesion (Sequence)

Una **sucesion** es una funcion f cuyo dominio es un subconjunto de los numeros enteros. Se escribe $a_n = f(n)$. Los valores de la funcion se llaman **terminos** de la sucesion. El dominio puede ser: los naturales $\{1,2,3,\dots\}$, los enteros no negativos $\{0,1,2,\dots\}$, los enteros entre dos valores dados, o los enteros mayores o iguales a un entero dado.

■ ANALOGIA — La sucesion es un Array con regla matematica

En programacion, un array es una lista de valores indexados: `arr[0]`, `arr[1]`, `arr[2]`, ... Una sucesion matematica es exactamente eso, pero con dos diferencias importantes: (1) el indice n es siempre un numero ENTERO (no existe la posicion 1.5 o 2.3), y (2) en vez de guardar los valores en memoria, existe una REGLA (la formula a_n) que calcula el valor de cualquier posicion directamente. Pensar en a_8 como llamar a una funcion: `getElement(8)`.

1.2 Vocabulario Esencial

■ Rosen — Cap. 2.4 — terminologia clave

Terminos, notacion y conceptos que aparecen en todos los ejercicios:

a_n	$f(n)$	Termino general (general term). El output de la sucesion en la posicion n . Es la 'regla de formacion'.
n	indice (index)	El input. Siempre entero. Puede arrancar en 0 o en 1 segun como se defina la sucesion.
a_1	primer termino	El valor de arranque. En sucesiones recursivas es el caso base obligatorio.
a_{n-1}	termino anterior	El output del paso previo. Aparece en las definiciones recursivas.
Finita	dominio finito	La sucesion tiene un ultimo termino: $a_1, a_2, \dots, a_n$.
Infinita	dominio infinito	La sucesion no termina: $a_1, a_2, a_3, \dots$ (con puntos suspensivos)
Explicita	$a_n = f(n)$	La formula calcula a_n directamente a partir de n . Sin historia previa.
Recursiva / Recurrente	$a_n = f(a_{n-1}, \dots)$	Cada termino depende de los anteriores. Siempre necesita uno o mas valores iniciales.

1.3 Sucesion Explicita — Calculo Directo

■ Rosen — Cap. 2.4 — Termino general explicito

En una definicion explicita, la formula $a_n = f(n)$ te da el valor de CUALQUIER termino simplemente sustituyendo n . No hace falta calcular los terminos anteriores.

EJEMPLO — Calcular a_3 y a_8 para $a_n = 2n - 1$

$$a_n = 2n - 1$$

$$a_3 = 2(3) - 1 = 5$$

$$a_8 = 2(8) - 1 = 15$$

Para $a_n = (-1)^n * (3 - 1/n)$:

$$a_3 = (-1)^3 * (3 - 1/3) = -1 * (8/3) = -8/3$$

$$a_8 = (-1)^8 * (3 - 1/8) = +1 * (23/8) = 23/8$$

■ TRAMPA COMUN — Radianes vs Grados

Si la formula contiene $\sin(n)$, $\cos(n)$ o $\tan(n)$ SIN el simbolo de grados ($^\circ$), el argumento esta en radianes. La calculadora debe estar en modo RAD. Si la deja en DEG, obtenes resultados completamente diferentes y erroneos.

1.4 Sucesion Recursiva — Logica con Estado

■ Rosen — Cap. 2.4 — Relaciones de recurrencia, Definicion 3

DEFINICION — Relacion de Recurrencia (Recurrence Relation)

Una ecuacion que expresa a_n en terminos de uno o mas terminos anteriores a_{n-1} , a_{n-2} , etc. Para que la sucesion quede completamente determinada, se deben especificar uno o mas **valores iniciales** (condiciones iniciales).

■ ANALOGIA — Recursion = funcion que se llama a si misma con estado previo

En programacion, una funcion recursiva necesita: (1) un CASO BASE para no entrar en loop infinito, y (2) una REGLA de como el caso n depende del caso $n-1$. Una sucesion recursiva es identica: los 'casos base' son los valores iniciales (a_0 , a_1), y la 'regla' es la relacion de recurrencia. Igual que en recursion, no podés calcular a_{10} directamente: tenés que ejecutar a_2 , a_3 , ..., a_9 primero.

EJEMPLO — Fibonacci — la recursion clasica

Definicion: $F_n = F_{n-2} + F_{n-1}$ para $n \geq 2$

Condiciones iniciales: $F_0 = 1$, $F_1 = 1$

$$F_2 = F_0 + F_1 = 1 + 1 = 2$$

$$F_3 = F_1 + F_2 = 1 + 2 = 3$$

$$F_4 = F_2 + F_3 = 2 + 3 = 5$$

$$F_5 = F_3 + F_4 = 3 + 5 = 8$$

Secuencia: 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, ...

■ TRAMPA COMUN — Signos negativos en sustitucion recursiva

Si la formula dice $a_n = a_{n-1} - a_{n-2}$ y $a_{n-2} = -3$, entonces: $a_n = a_{n-1} - (-3) = a_{n-1} + 3$. SIEMPRE usar parentesis al sustituir valores negativos.

PARTE 2 — TIPOS DE SUCESION

2.1 Progresion Aritmetica (Arithmetic Progression)

■ Rosen — Cap. 2.4 — Ejemplo 5, pagina 163

DEFINICION — Sucesion Aritmetica

Una sucesion donde la diferencia entre dos terminos consecutivos cualesquiera es siempre la misma constante **d**, llamada **diferencia comun** (common difference). Formalmente: $a_{n+1} - a_n = d$ para todo n .

■ ANALOGIA — Contador con paso fijo — como un for loop

Un `for(i=1; i<=n; i+=d)` genera exactamente una progresion aritmetica: cada iteracion suma el mismo 'd' al valor anterior. El primer valor del loop (i=1) es a_1 , el paso es d. Si $d > 0$ la secuencia crece; si $d < 0$ decrece. Ej: `for(i=100; i>=10; i-=9)` genera 100, 91, 82, 73, 64, 55, 46, 37... (d = -9)

Termino general:	$a_n = a_1 + d(n - 1)$
Diferencia:	$d = a_{n+1} - a_n$ (cualquier par consecutivo)
Forma recursiva:	$a_n = a_{n-1} + d$ para $n > 1$
Serie SA (suma):	$S_n = (a_1 + a_n) / 2 \cdot n$

EJEMPLO — Identificar y formular 10, 17, 24, 31, 38...

Test: $17-10 = 7$, $24-17 = 7$, $31-24 = 7 \rightarrow d = 7$ (constante) ✓

$a_1 = 10$, $d = 7$

$a_n = 10 + 7(n-1) = 10 + 7n - 7 = 7n + 3$

Verificacion: $a_1 = 7(1)+3 = 10$ ✓ $a_3 = 7(3)+3 = 24$ ✓

2.2 Progresion Geometrica (Geometric Progression)

■ Rosen — Cap. 2.4 — Ejemplo 6, pagina 163

DEFINICION — Sucesion Geometrica

Una sucesion donde el cociente entre dos terminos consecutivos cualesquiera es siempre la misma constante **q**, llamada **razon comun** (common ratio). Formalmente: $a_{n+1} / a_n = q$ para todo n .

■ ANALOGIA — Crecimiento exponencial — duplicar, triplicar, halvar

Una progresion geometrica modela cualquier proceso donde algo se MULTIPLICA por un factor fijo: bacterias que se duplican cada hora ($q=2$), capital que crece al 10% anual ($q=1.1$), señal de radio que pierde la mitad de intensidad por kilometro ($q=0.5$). La clave: no se SUMA una constante, se MULTIPLICA una constante.

Termino general (a_1):	$a_n = a_1 \cdot q^{n-1}$
Termino general (a_0):	$a_n = a_0 \cdot q^n$
Razon:	$q = a_{n+1} / a_n$ (cualquier par consecutivo)
Serie SG finita:	$S_n = (1 - q^n) / (1 - q) \cdot a_1$
Serie SG infinita:	$S_{inf} = a_1 / (1 - q)$ solo si $ q < 1$

EJEMPLO — Identificar y formular 4, 12, 36, 108, 324...

Test resta: $12-4=8$, $36-12=24 \rightarrow$ NO es constante. No es aritmetica.

Test division: $12/4=3$, $36/12=3$, $108/36=3 \rightarrow q = 3$ (constante) ✓

$a_1 = 4$, $q = 3$

$a_n = 4 \cdot 3^{(n-1)}$

Verificacion: $a_1 = 4 \cdot 3^0 = 4$ ✓ $a_4 = 4 \cdot 3^3 = 4 \cdot 27 = 108$ ✓

2.3 Sucesion Cuadratica y Cubica

■ Rosen — Extension del metodo de diferencias finitas — no en Rosen directamente

DEFINICION — Sucesion Cuadratica / Cubica

Una secuencia tiene patron cuadratico cuando la sucesion de SEGUNDA DIFERENCIA es constante. Tiene patron cubico cuando la TERCERA DIFERENCIA es constante. En ambos casos el termino general es un polinomio de grado 2 o 3 respectivamente.

■ ANALOGIA — Derivadas discretas — diferencias finitas

En calculo continuo, la primera derivada mide la tasa de cambio; la segunda derivada mide la curvatura. En matematica discreta hacemos lo mismo con sumas: la 'primera diferencia' ($d_i = a_{i+1} - a_i$) es la derivada discreta. Si la primera diferencia es constante $\rightarrow$ lineal (aritmetica). Si la segunda diferencia (diferencia de las diferencias) es constante $\rightarrow$ cuadratica. Si la tercera diferencia es constante $\rightarrow$ cubica. Es exactamente como el grado de derivacion que necesitas para aplanar la curva.

CUADRATICA $a_n = An^2 + Bn + C$

Paso 1: encontrar A

$$2A = d' \text{ (2a diferencia constante)}$$

Paso 2: encontrar B

$$3A + B = d'_1 \text{ (1a diferencia inicial)}$$

Paso 3: encontrar C

$$A + B + C = a_1$$

CUBICA $a_n = An^3 + Bn^2 + Cn + D$

Paso 1: encontrar A

$$6A = d'' \text{ (3a diferencia constante)}$$

Paso 2: encontrar B

$$12A + 2B = d''_1 \text{ (2a diferencia inicial)}$$

Paso 3: encontrar C

$$7A + 3B + C = d'_1 \text{ (1a diferencia inicial)}$$

Paso 4: encontrar D

$$A + B + C + D = a_1$$

EJEMPLO — Encontrar a_n para 2, 6, 12, 20, 30...

1a diferencias: $6-2=4$, $12-6=6$, $20-12=8$, $30-20=10 \rightarrow [4, 6, 8, 10]$

2a diferencias: $6-4=2$, $8-6=2$, $10-8=2 \rightarrow d'=2$ (constante) $\rightarrow$ CUADRATICA

Paso 1: $2A = d' = 2 \rightarrow A = 1$

Paso 2: $3A + B = d'_1 = 4 \rightarrow 3(1) + B = 4 \rightarrow B = 1$

Paso 3: $A + B + C = a_1 = 2 \rightarrow 1 + 1 + C = 2 \rightarrow C = 0$

Resultado: $a_n = n^2 + n$

Verificacion: $a_1=1+1=2$ ✓ $a_3=9+3=12$ ✓ $a_5=25+5=30$ ✓

2.4 Excepciones: Motor + Offset y Fracciones

Cuando las diferencias EXPLOTAN (crecen sin parar) o son ellas mismas geometricas, la secuencia tiene un componente de crecimiento rapido llamado MOTOR.

Motor tipico 2^n :	$a_n = 2^n + \text{offset} \rightarrow$ comparas motor con serie real, diferencia = offset
Motor 3^n :	$a_n = 3^n + k$
Motor $n!$:	$a_n = n! + k$
Fracciones:	Separa: $a_n(\text{num})/a_n(\text{den}) \rightarrow$ encuentra a_n del numerador y b_n del denominador
Alternante:	$(-1)^n$ o $(-1)^{(n+1)} \rightarrow$ controla el signo (+ o -) en cada posicion

EJEMPLO — Motor+Offset: 5, 7, 11, 19, 35, 67...

1a diferencias: 2, 4, 8, 16, 32 $\rightarrow$ geometricas! $q=2 \rightarrow$ motor es 2^n

Comparar motor 2^n con la serie:

$n=1$: $2^1=2$ vs $a_1=5 \rightarrow$ diferencia = 3

$n=2$: $2^2=4$ vs $a_2=7 \rightarrow$ diferencia = 3

$n=3$: $2^3=8$ vs $a_3=11 \rightarrow$ diferencia = 3 $\rightarrow$ offset = 3 (constante) ✓

Resultado: $a_n = 2^n + 3$

Verificacion: $a_5 = 2^5+3 = 32+3 = 35$ ✓

PARTE 3 — SERIES Y SUMATORIAS

3.1 Que es una Serie

■ Rosen — Cap. 2.4 — Sumatorias, pagina 165

DEFINICION — Serie (Series / Summation)

Una **serie** es la suma de los terminos de una sucesion. Si la sucesion es finita, la serie es finita. Si la sucesion es infinita, la serie es infinita. La **notacion sigma** (simbolo griego S mayuscula, letra 's' de 'suma') es la forma concisa de escribir estas sumas.

■ ANALOGIA — La Sigma es un bucle 'for' que acumula

La expresion $\sum_{i=m}^n f(i)$ es exactamente: `total = 0; for(i=m; i<=n; i++) { total += f(i); } return total;` El indice 'i' es la variable del loop, 'm' es el limite inferior (valor inicial del loop), 'n' es el limite superior (condicion de parada), y 'f(i)' es el cuerpo del loop. Importante: el loop **INCLUYE** el valor n (es `<=`, no `<`).

3.2 Vocabulario de Sumatorias

Sigma S	$\sum_{i=m}^n$	El simbolo de suma. Lee: 'sumatoria de i igual a m hasta n'.
Indice (index)	i (o j, k, etc.)	La variable del bucle. Puede llamarse i, j, k. Solo importa que sea consistente.
Limite inferior	m (valor debajo de S)	Donde empieza el bucle. Puede ser 0, 1, o cualquier entero.
Limite superior	n (valor arriba de S)	Donde termina el bucle (inclusive). Si es 'inf' la suma es infinita.
Termino general	a_i o $f(i)$	La expresion que se evalua y acumula en cada paso del loop.
forEach (E S)	$\sum_{j \in S}$	j itera sobre el conjunto S solamente. No va de 0 a n secuencial.

EJEMPLO — Diferencia entre for clasico y forEach

For clasico: $\sum_{i=1}^4 i^2 = 1^2 + 2^2 + 3^2 + 4^2 = 1+4+9+16 = 30$

forEach: $\sum_{j \in \{1,3,5,7\}} j^2 = 1^2 + 3^2 + 5^2 + 7^2 = 1+9+25+49 = 84$

La diferencia: el for clasico va 1,2,3,4 (todos los enteros).

El forEach va 1,3,5,7 (solo los elementos del conjunto S).

3.3 Propiedades de Sumatorias

■ Rosen — Cap. 2.4 — Tabla 2, pagina 166

Aditividad	$S(a_n + b_n) = Sa_n + Sb_n$	Si adentro hay suma o resta → separar en dos sumatorias independientes. Como distribuir un for sobre dos acumuladores.
Escalar	$S(c \cdot a_n) = c \cdot Sa_n$	Una constante que NO depende del indice sale afuera de la sumatoria. c es constante si no tiene 'i' o 'n' en su expresion.

Rango partido

$$S_{i=m}^n = S_{i=m}^j + S_{i=j+1}^n$$

Podes partir un loop en dos loops consecutivos que en conjunto cubren el mismo rango.

Invertir orden anidada

$$S_i S_j = S_j S_i$$

En una doble sumatoria, podes resolver primero el loop que te resulte mas comodo.

■ TRAMPA COMUN — Dos errores que no se pueden cometer

ERROR 1: $S(a \cdot b) = S a \cdot S b$ — FALSO. La sumatoria NO se distribuye en multiplicaciones. Solo se distribuye en sumas y restas. ERROR 2: $S_{i=1}^3 5 = 5$ — FALSO. Una constante sumada n veces ES n.c, no c. $S_{i=1}^3 5 = 5+5+5 = 15 = 5.3$.

3.4 Sumatorias Anidadas

■ Rosen — Cap. 2.4 — Doble sumatoria

Cuando hay dos sigmas juntas SS, se trata de un bucle dentro de otro. La regla es siempre: resolver DE ADENTRO HACIA AFUERA.

■ ANALOGIA — Double for loop — el de afuera es constante mientras el de adentro corre

for(i=1; i<=3; i++) { for(j=1; j<=2; j++) { total += (j - i); } } Mientras el loop externo tiene i fijo (digamos i=2), el loop interno corre j=1,2. i=2 es una CONSTANTE durante ese pasaje. Una vez que el loop interno termina, i avanza a 3 y el loop interno vuelve a empezar. En matematica: congela la variable externa, resuelve la sumatoria interna, luego inyecta ese resultado en la sumatoria externa.

EJEMPLO — Resolver $S_{i=1 \rightarrow 3} S_{j=1 \rightarrow 2} (j - i)$

Paso 1 (Scope interno): congelar i, resolver $S_{j=1 \rightarrow 2} (j - i)$

Con i=1: $(1-1) + (2-1) = 0 + 1 = 1$

Con i=2: $(1-2) + (2-2) = -1 + 0 = -1$

Con i=3: $(1-3) + (2-3) = -2 + (-1) = -3$

Paso 2 (Scope externo): sumar los resultados del loop interno

$S_{i=1 \rightarrow 3} \text{resultado_interno} = 1 + (-1) + (-3) = -3$

Respuesta: -3

3.5 Formulas Cerradas — Las 7 Esenciales

■ Rosen — Cap. 2.4 — Tabla 2, pagina 166

Estas formulas permiten calcular una sumatoria sin sumar termino por termino. En la induccion matematica van a aparecer constantemente.

$S 1 (i=1 \text{ a } n)$	$= n$
$S i (i=1 \text{ a } n)$	$= n(n+1)/2$
$S i^2 (i=1 \text{ a } n)$	$= n(n+1)(2n+1)/6$
$S i^3 (i=1 \text{ a } n)$	$= [n(n+1)/2]^2$
$S r^i (i=0 \text{ a } n)$	$= (r^{n+1} - 1) / (r - 1) \text{ para } r \neq 1$
$SA S_n (n \text{ terminos})$	$= (a_1 + a_n) / 2 \cdot n$
$SG S_n (n \text{ terminos})$	$= (1 - q^n) / (1 - q) \cdot a_1$
$SG S_{\infty} (\text{infinita})$	$= a_1 / (1 - q) \text{ solo cuando } q < 1 \text{ (converge)}$

■ ANALOGIA — Por que $S[i=1 \text{ a } n] = n(n+1)/2$ — la historia de Gauss

Gauss, de niño, necesitaba sumar $1+2+3+\dots+100$. Observo que si escribis la suma al derecho y al revés y las sumas en pares: $(1+100)+(2+99)+(3+98)+\dots = 101 \cdot 100/2 = 5050$. Eso es exactamente la fórmula $n(n+1)/2$. En programación, tener esta fórmula evita un loop $O(n)$ y lo reemplaza por $O(1)$. Por eso son tan valiosas: colapsan un for loop en una sola operación.

3.6 Productoria (Π) y Factorial

■ Rosen — Cap. 2.4 — Productos

DEFINICION — Productoria (Product Notation)

El símbolo Pi (Π) es la versión multiplicativa de la Sigma. $\prod_{j=m}^n a_j = a_m \cdot a_{m+1} \cdot \dots \cdot a_n$. Es un bucle que MULTIPLICA en vez de sumar.

DEFINICION — Factorial ($n!$)

El factorial de un entero positivo n es el producto de todos los enteros de 1 a n : $n! = n \cdot (n-1) \cdot (n-2) \cdot \dots \cdot 3 \cdot 2 \cdot 1$. Se define $0! = 1$ como caso base. Forma recursiva: $n! = n \cdot (n-1)!$ para $n \geq 1$.

Notación Π	$\prod_{j=m \rightarrow n} a_j = a_m \cdot a_{m+1} \cdot \dots \cdot a_n$
Factorial	$n! = \prod_{i=1 \rightarrow n} i = 1 \cdot 2 \cdot 3 \cdot \dots \cdot n$
Caso base	$0! = 1$ (por definición, no por cálculo)
Recursiva	$n! = n \cdot (n-1)!$ para $n \geq 1$
HACK CLAVE	$k! \cdot (k+1) = (k+1)! \leftarrow$ usar en inducción con factoriales

■ TRAMPA COMUN — Convergencia — cuando una serie infinita tiene valor finito

La serie geométrica infinita $S_{\text{inf}} = a_1/(1-q)$ SOLO tiene sentido (solo converge) cuando $|q| < 1$. Si $|q| \geq 1$, los términos no tienden a 0 y la suma diverge (crece sin límite o alterna sin estabilizarse). Intuición: si cada término es menor que el anterior por un factor fijo < 1 , la suma total queda acotada.

PARTE 4 — INDUCCION MATEMATICA

4.1 El Principio — Que es y Para Que Sirve

■ Rosen — Cap. 5.1 — Principio de Induccion Matematica, pagina 331

DEFINICION — Principio de Induccion Matematica (PMI)

Sea $P(n)$ una proposicion definida para enteros $n \geq a$. Si se cumplen DOS condiciones: (1) $P(a)$ es verdadera [PASO BASE], y (2) para todo entero $k \geq a$, si $P(k)$ es verdadera entonces $P(k+1)$ tambien es verdadera [PASO INDUCTIVO], ENTONCES $P(n)$ es verdadera para todo entero $n \geq a$.

■ ANALOGIA — Las fichas de domino que caen — una por una, para siempre

Imagine una fila infinita de fichas de domino. Para probar que TODAS van a caer, solo necesitas demostrar DOS cosas: (1) la PRIMERA ficha cae (Paso Base), y (2) si CUALQUIER ficha cae, la siguiente tambien cae (Paso Inductivo). Con esas dos condiciones, la cadena es imparable: 1 cae $\rightarrow$ 2 cae $\rightarrow$ 3 cae $\rightarrow$... $\rightarrow$ infinito. En programacion, es como probar que un algoritmo recursivo es correcto para todo n : probas el caso base $n=1$ y luego probas que 'si funciona para k , funciona para $k+1$ '.

4.2 Vocabulario Tecnico de Induccion

■ Rosen — Cap. 5.1 — terminologia, pagina 332

P(n)	proposicion / predicado	El enunciado que queremos demostrar. Puede ser una igualdad, desigualdad, o divisibilidad.
Paso Base (Basis Step)	verificar $P(a)$	Probar que el enunciado es cierto para el primer valor (generalmente $n=1$ o $n=0$).
Hipotesis Inductiva (HI)	asumir $P(k)$	Suponer que $P(k)$ es verdadero para algun entero $k \geq a$. Es la herramienta principal.
Tesis Inductiva	demostrar $P(k+1)$	Lo que hay que llegar a probar. Escribis $P(k+1)$ limpia y es tu TARGET.
Paso Inductivo	$HI \rightarrow Tesis$	La demostracion: partiendo de algun lado, usando la HI como herramienta, llegar a la Tesis.
Conclusion	n -esimo paso	'Por el PMI, $P(n)$ es verdadero para todo entero $n \geq a$.' Cierra con cuadrado ■.

4.3 Los 5 Pasos — El Algoritmo Universal

Sin importar el tipo de ejercicio, la estructura de la demostracion es SIEMPRE esta:

PASO BASE

- 1 Sustituir $n = a$ (generalmente $n=1$). Calcular ambos lados y verificar que son iguales (o que la desigualdad se cumple). Si da Falso $\rightarrow$ el enunciado esta mal. Si da Verdadero $\rightarrow$ avanzar.

HIPOTESIS INDUCTIVA

- 2 Escribir: 'Supongamos que $P(k)$ es verdadero para algun entero $k \geq a$.' Reemplazar n por k en toda la proposicion. Este enunciado es tu herramienta principal.

TESIS INDUCTIVA

- 3** Escribir $P(k+1)$ de forma limpia: reemplazar n por $k+1$ en toda la proposición. CONVENCION: hacer las distributivas de $(k+1)^2$ etc. en un borrador para saber exactamente a donde apuntar.

DEMOSTRACION

- 4** Partir de un lado de la Tesis. Inyectar la HI en algun punto (donde aparece el bloque que reconoces de la HI). Hacer algebra hasta llegar al otro lado. Factor Comun ANTES de expandir. El ultimo termino de una sumatoria se calcula con la formula original en $k+1$.

CONCLUSION

- 5** Escribir la conclusion formal: 'Como se verifico el Paso Base y el Paso de Induccion, por el Principio de Induccion Matematica, $P(n)$ es verdadero para todo entero $n \geq a$. ■'

4.4 Patron 1 — Igualdad (Sumatorias = Formula Cerrada)

■ Rosen — Cap. 5.1 — Ejemplo 1, pagina 333

Reconoces este patron cuando el enunciado tiene un SIGNO IGUAL con una sumatoria a la izquierda y una formula cerrada a la derecha.

EJEMPLO — Demostrar: $1+2+3+\dots+n = n(n+1)/2$ para todo $n \geq 1$

$P(n)$: $S[i=1 \text{ a } n] \ i = n(n+1)/2$

PASO 1 — Base ($n=1$):

LHS: $S[i=1 \text{ a } 1] \ i = 1$

RHS: $1(1+1)/2 = 2/2 = 1$

LHS = RHS = 1 ✓

PASO 2 — Hipotesis Inductiva (HI):

Supongamos $P(k)$ verdadero: $S[i=1 \text{ a } k] \ i = k(k+1)/2$

PASO 3 — Tesis Inductiva:

$P(k+1)$: $S[i=1 \text{ a } k+1] \ i = (k+1)(k+2)/2$

PASO 4 — Demostracion (partimos del LHS de la Tesis):

$S[i=1 \text{ a } k+1] \ i = S[i=1 \text{ a } k] \ i + (k+1) \leftarrow$ separar ultimo termino

$= k(k+1)/2 + (k+1) \leftarrow$ inyectar HI

$= (k+1) [k/2 + 1] \leftarrow$ factor comun $(k+1)$

$= (k+1) \cdot (k+2)/2 = (k+1)(k+2)/2 \checkmark$

PASO 5 — Conclusion:

Por el PMI, $S[i=1 \text{ a } n] \ i = n(n+1)/2$ para todo entero $n \geq 1$. ■

4.5 Patron 2 — Desigualdad

■ Rosen — Cap. 5.1 — Ejemplo 5, pagina 337

Reconoces este patron cuando el enunciado tiene $<$ o $>$ en lugar de $=$. La estrategia cambia: al inyectar la HI, el signo igual se convierte en desigualdad.

EJEMPLO — Demostrar: $2^n > n^2$ para todo $n > 4$ (Rosen Ejemplo 5)

$P(n)$: $2^n > n^2$

PASO 1 — Base ($n=5$, porque el enunciado dice $n>4$):

LHS: $2^5 = 32$

RHS: $5^2 = 25$

$32 > 25$ ✓

PASO 2 — HI: Supongamos $2^k > k^2$ para algún $k \geq 5$

PASO 3 — Tesis: demostrar $2^{(k+1)} > (k+1)^2$

PASO 4 — Demostración:

$2^{(k+1)} = 2 \cdot 2^k$

$> 2 \cdot k^2 \leftarrow$ por la HI ($2^k > k^2$, multiplicamos por 2)

$= k^2 + k^2$

$\geq k^2 + 4k \leftarrow$ porque $k \geq 5$, entonces $k^2 \geq 4k$

$> k^2 + 2k + 1 \leftarrow$ porque $4k > 2k+1$ para $k \geq 2$

$= (k+1)^2$ ✓

PASO 5: Por el PMI, $2^n > n^2$ para todo $n > 4$. ■

4.6 Patron 3 — Divisibilidad

■ Rosen — Cap. 5.1 — Ejemplo 8

Reconoces este patron cuando el enunciado dice 'es divisible por X', 'es multiplo de X', o usa la barra vertical: $X \mid f(n)$. La clave es que la HI y la Tesis siempre se expresan como multiplos de X, con LETRAS DISTINTAS.

EJEMPLO — Demostrar: $3 \mid (n^3 + 2n)$ para todo entero $n \geq 1$

$P(n)$: $n^3 + 2n$ es divisible por 3

PASO 1 — Base ($n=1$):

$1^3 + 2(1) = 3 = 3 \cdot 1 \rightarrow$ divisible por 3 ✓

PASO 2 — HI: $k^3 + 2k = 3q$ para algun entero q

PASO 3 — Tesis: $(k+1)^3 + 2(k+1)$ es divisible por 3

PASO 4 — Demostración (Tetris algebraico):

$(k+1)^3 + 2(k+1)$

$= k^3 + 3k^2 + 3k + 1 + 2k + 2 \leftarrow$ expandir $(k+1)^3$

$= (k^3 + 2k) + 3k^2 + 3k + 3 \leftarrow$ agrupar HI

$= 3q + 3k^2 + 3k + 3 \leftarrow$ inyectar HI

$= 3(q + k^2 + k + 1) \leftarrow$ factor comun 3

Como $(q + k^2 + k + 1)$ es entero, el resultado es divisible por 3 ✓

PASO 5: Por el PMI, $3 \mid (n^3 + 2n)$ para todo $n \geq 1$. ■

■ TRAMPA COMUN — Scope de variables en divisibilidad

En la HI escribis: $f(k) = X \cdot q$. En la Tesis necesitas demostrar que $f(k+1) = X \cdot t$ (con una letra DIFERENTE, por ej. 't' o 's'). Si usas la misma letra estás asumiendo que $q = t$, lo cual es exactamente lo que querés demostrar y convierte la prueba en circular. Scope separado, letras separadas.

4.7 Patron 4 — Productoria

Identico al patron de Igualdad pero el acumulador es de multiplicacion (Π). En vez de separar el 'ultimo termino', separas el 'ultimo FACTOR'.

EJEMPLO — Demostrar: $P[k=2 \text{ a } n] (1 - 1/k) = 1/n$ para todo $n \geq 2$

$P(n): P[k=2 \rightarrow n] (1 - 1/k) = 1/n$

PASO 1 — Base ($n=2$):

$P[k=2 \rightarrow 2] (1 - 1/k) = (1 - 1/2) = 1/2$

RHS: $1/2 \checkmark$

PASO 2 — HI: $P[k=2 \rightarrow k] (1 - 1/k) = 1/k$ (usando 'k' como variable libre tambien)

Mas claro: $P[j=2 \rightarrow m] (1 - 1/j) = 1/m$ para algun $m \geq 2$

PASO 3 — Tesis: $P[j=2 \rightarrow (m+1)] (1 - 1/j) = 1/(m+1)$

PASO 4 — Demostracion:

$P[j=2 \rightarrow m+1] (1 - 1/j)$

$= P[j=2 \rightarrow m] (1 - 1/j) \cdot (1 - 1/(m+1)) \leftarrow$ separar ultimo factor

$= (1/m) \cdot (m/(m+1)) \leftarrow$ inyectar HI

$= m / (m \cdot (m+1))$

$= 1/(m+1) \checkmark$

PASO 5: Por el PMI, $P[k=2 \rightarrow n] (1 - 1/k) = 1/n$ para todo $n \geq 2$. ■

PARTE 5 — GLOSARIO BILINGUE CON ANALOGIAS

Todos los terminos tecnicos del tema con su nombre en ingles (como en Rosen), su definicion formal y su analogia de programacion.

Sequence / Sucesion	$f: \mathbb{Z}_{\text{subconjunto}} \rightarrow \mathbb{R}$	Array matematico con indice entero y regla de calculo.
Term / Termino	$a_n = f(n)$	El valor del array en la posicion n. Como arr[n].
Index / Indice	$n, i, k, j \dots$	La variable de posicion. Siempre entero, puede empezar en 0 o 1.
General Term / Termino General	a_n o $f(n)$	La regla o formula que genera todos los terminos. Como la funcion del generador.
Recurrence Relation / Relacion de Recurrencia	$a_n = f(a_{n-1}, \dots)$	Funcion recursiva: el output depende de outputs anteriores.
Initial Condition / Condicion Inicial	$a_0 = c$ o $a_1 = c$	El caso base de la recursion. Sin el, el loop no puede arrancar.
Arithmetic Progression / Progresion Aritmetica	$d = a_{n+1} - a_n = \text{constante}$	For loop con incremento fijo: $i += d$ en cada iteracion.
Common Difference / Diferencia Comun	d	El 'paso' del for loop aritmetico.
Geometric Progression / Progresion Geometrica	$q = a_{n+1} / a_n = \text{constante}$	Multiplicacion por factor fijo en cada iteracion: $i *= q$.
Common Ratio / Razon Comun	q	El factor multiplicativo del for geometrico.
Series / Serie	$\sum a_i$	Acumulador que suma todos los terminos. Variable total que crece en el loop.
Summation / Sumatoria	$\sum_{i=m \rightarrow n} a_i$	Sintaxis concisa para el for loop acumulador. $\sum$ = sigma = sum.
Index of Summation	i (dentro de $\sum$)	La variable del for loop. Solo existe dentro del scope del sigma.
Lower Limit / Limite Inferior	m (debajo de $\sum$)	El valor inicial del for: <code>int i = m</code> .
Upper Limit / Limite Superior	n (arriba de $\sum$)	La condicion de parada del for: <code>i <= n</code> .
Product Notation / Productoria	$\prod_{j=m \rightarrow n} a_j$	For loop que MULTIPLICA en vez de sumar. Acumulador con <code>total *= a_j</code> .
Factorial	$n! = 1.2.3 \dots n$	$\prod_{i=1 \rightarrow n} i$. El producto de todos los enteros de 1 a n. $0! = 1$ por convencion.
Mathematical Induction / Induccion Matematica	$P(1) \wedge (P(k) \rightarrow P(k+1)) \implies \forall n P(n)$	Prueba que un algoritmo recursivo es correcto: caso base + paso inductivo.
Basis Step / Paso Base	Verificar $P(1)$ o $P(a)$	Unit test del caso inicial. Si falla, el enunciado es falso.
Inductive Hypothesis / Hipotesis Inductiva	Asumir $P(k)$ verdadero	Declarar una 'variable axiomatica': <code>assert P(k) == True</code> . Es la herramienta.
Inductive Step / Paso Inductivo	$P(k) \rightarrow P(k+1)$	Probar que si el algoritmo funciona para $n=k$, funciona para $n=k+1$.

Conjecture / Conjetura	Hipotesis no demostrada	Un pattern que observas en los datos pero que todavia no probaste formalmente.
Divisible / Divisible	$X \mid f(n)$	$f(n) \bmod X == 0$. El resultado es multiplo entero de X.
Convergence / Convergencia	$\lim_{n \rightarrow \infty} a_n = L$	La serie tiende a un valor finito. Ocurre cuando $ q < 1$ en SG.
Divergence / Divergencia	$\lim_{n \rightarrow \infty} a_n = \infty$ o no existe	La serie no tiene limite finito. Ocurre cuando $ q \geq 1$.